```
// SPDX-License-Identifier: MIT

pragma solidity ^0.6.12;

contract Banking {

    mapping(address => uint256) public user_account;
    mapping(address => bool) public user_exists;

    // Create Account
    function create_account() public payable returns (string memory) {

        require(!user_exists[msg.sender], "Account already created");

        user_exists[msg.sender] = true;
        user_account[msg.sender] = msg.value;

        return "Account created";
    }

    // Deposit Amount
    function deposit() public payable returns (string memory) {

        require(user_exists[msg.sender], "Account not created");
        require(msg.value > 0, "Deposit amount should be greater than zero");

        user_account[msg.sender] += msg.value;

        return "Deposited Successfully";
    }
```

```
// Withdraw Amount
function withdraw(uint256 amount) public returns (string memory) {

    require(user_exists[msg.sender], "Account not created");
    require(amount > 0, "Amount should be greater than zero");
    require(user_account[msg.sender] >= amount, "Insufficient Balance");

    user_account[msg.sender] -= amount;
    msg.sender.transfer(amount);

    return "Withdrawal Successful";
}

// Transfer within Bank
function transfer(address userAddress, uint256 amount)
    public
    returns (string memory)
{

    require(user_exists[msg.sender], "Account not created");
    require(user_exists[userAddress], "Receiver account not created");
    require(amount > 0, "Amount should be greater than zero");
    require(user_account[msg.sender] >= amount, "Insufficient Balance");

    user_account[msg.sender] -= amount;
    user_account[userAddress] += amount;

    return "Transfer Successful";
}
```

```
    // Send Ether to External Address
    function send_amt(address payable toAddress, uint256 amount)
        public
        returns (string memory)
    {

        require(user_exists[msg.sender], "Account not created");
        require(amount > 0, "Amount should be greater than zero");
        require(user_account[msg.sender] >= amount, "Insufficient Balance");

        user_account[msg.sender] -= amount;
        toAddress.transfer(amount);

        return "Transfer Successful";
    }

    // Check Balance
    function user_balance() public view returns (uint256) {

        return user_account[msg.sender];
    }

    // Check Account Status
    function account_exist() public view returns (bool) {

        return user_exists[msg.sender];
    }
}
```